



LO02 - compte-rendu de fin de projet JEST

**Raphael BEAU
Taha MACHICHI EL IDRISI**

Différences entre les diagrammes

les deux diagrammes se trouvent dans le répertoire du projet dans `diagramme/`

Nouvelles classes

La classe Jest

Dans le premier jet, le joueur avait juste une liste de cartes de score. Maintenant, il possède un objet Jest dédié.

La raison du changement : C'est une question de responsabilité. Le calcul du score dans ce jeu est complexe (bonus de couleurs, suites, trophées). En créant une classe Jest, on lui donne la responsabilité de gérer l'interaction avec le ScoreVisitor. Le Joueur reste ainsi une classe simple qui "possède" des choses, tandis que le Jest devient l'expert du stockage des points.

La classe Offre

Auparavant, les cartes proposées par les joueurs étaient mélangées dans leur main ou dans une liste générique.

La raison du changement : Dans les règles du jeu, l'offre a un état très particulier (une carte face visible, une carte face cachée). Créer une classe Offre permet d'encapsuler cette logique de visibilité. Cela évite d'avoir des `if (carte.estVisible)` partout dans la classe Joueur.

La classe ScannerHybride

Elle n'existait pas du tout au début ; elle est née de la nécessité de faire cohabiter la console et l'interface graphique.

La raison du changement : C'est le pont de l'application. Permet à tout le moteur de jeu pour qu'il "écoute" soit le clavier, soit les clics de souris. Le ScannerHybride unifie ces deux mondes : le jeu croit lire du texte, mais ce texte peut provenir d'un bouton de la VueGraphique.

Les classes ScoreVisitorCameleon et ScoreVisitorMiroir (Spécialisations)

Au départ, il n'y avait qu'un seul visiteur de score de base.

La raison du changement : C'est l'application directe du principe d'Extension. Plutôt que de modifier le code de calcul existant à chaque fois que l'on ajoute une extension au jeu (Cameleon ou Miroir), on crée une nouvelle classe qui hérite de ScoreVisitorBase. Cela permet de changer la règle de calcul à la volée selon l'option choisie au menu sans toucher au reste du programme.

La classe Trophée

Les trophées n'étaient pas représentés comme des objets à part entière au départ.

La raison du changement : À la fin du jeu, les trophées sont distribués selon des conditions (celui qui a le plus de pique, la plus grosse valeur, etc.). En faire une classe permet de définir une "condition de victoire" propre à chaque trophée. C'est plus propre que de mettre toute cette logique dans la classe Jeu.

Les classes de Stratégies (StrategieSimple, StrategieAleatoire)

Dans le premier diagramme, le JoueurVirtuel était une boîte noire.

Pour pouvoir changer l'intelligence de l'IA sans modifier la classe JoueurVirtuel. C'est le patron Strategy.

La classe SauvegardeJeu

Elle est apparue avec l'implémentation de la persistance des données.

La raison du changement : Pour respecter le principe de Responsabilité Unique. Le Jeu s'occupe de jouer, pas d'écrire sur le disque dur. Cette classe gère la sérialisation (l'enregistrement) et le chargement des fichiers pour que l'on puisse reprendre une partie plus tard. Dans le diagramme d'origine, quelque chose de plus simple était montré car nous n'avions pas encore eu les cours sur la sérialisation.

La classe VueComposite

Dans le premier diagramme, il y avait une VueConsole et une VueGraphique, mais le Jeu devait choisir l'une ou l'autre.

La raison du changement (Design Pattern Composite) : Cette classe permet de regrouper plusieurs vues en une seule. Le Jeu envoie ses informations à la VueComposite, qui les distribue simultanément à la console et à la fenêtre graphique. Cela permet d'avoir les deux affichages en même temps sans que le code du moteur de jeu ne devienne complexe.

Le patron MVC

Le Modèle (Jeu, Joueur, Carte) : Ce sont les données. Ils ne savent pas si tu joues sur une console ou une fenêtre Windows. Ils gèrent uniquement les règles et les scores.

La Vue (VueConsole, VueGraphique, VueComposite) : C'est l'affichage pur. Elles reçoivent les données du modèle et les montrent à l'utilisateur. Elles ne prennent aucune décision logique.

Le Contrôleur (Controleur) : C'est le chef d'orchestre. Il reçoit les ordres de la Vue (clics, touches clavier) et dit au Modèle ce qu'il doit faire.

Etat du projet

À l'issue de cette période de développement, nous avons le plaisir d'annoncer que le projet JEST est désormais terminé avec succès. Nous avons réussi à traduire l'intégralité des règles du jeu de cartes physique en une application logicielle robuste, tout en respectant l'ensemble des contraintes techniques imposées.

En conclusion, ce projet nous a permis de démontrer notre capacité à concevoir une application complexe, évolutive et centrée sur l'utilisateur. Nous considérons que les objectifs sont pleinement atteints et que le logiciel est prêt pour une utilisation finale, répondant point par point aux exigences de qualité et de performance du cahier des charges.